Data Models

Conceptual Modelling

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topic 1

Topic 1 gave us a decision framework: **identify the workload → match to model**.

The relational model won several scenarios — banking, orders, inventory.

But choosing the right model is only half the problem. The next question: **how do you design a correct relational schema?**

ShopEasy: The Payment Question Nobody Could Answer

ShopEasy is an e-commerce platform processing ~800 orders per day.

Six months after launch, the product team asks: *can a customer pay for a single order in instalments?*

The developer opens the schema:

```
CREATE TABLE Payment (
    payment_id   INT PRIMARY KEY,
    order_id     INT REFERENCES Order_(order_id),
    amount       DECIMAL(10,2),
    payment_date DATE,
    method       VARCHAR(30)
);
```

No `UNIQUE` on `order_id`. No `NOT NULL`. The schema accepts one payment or many — and nobody can tell which was intended.

ShopEasy: The Cost of Ambiguity

The checkout flow inserts one payment per order. The refund-and-repay flow inserts a second. Both work without error.

The business rule — "every payment must be linked to exactly one order" — was clear in the requirements document.

But between the requirements and `CREATE TABLE`, the rule got lost.

Cost: two-week investigation, risky migration on 1.4 million payment records, a defect that reached production.

The schema was technically correct. But it could not answer: **what did we intend?**

UniTrack: The Enrolment Ghost

UniTrack is a university registration system. The developer writes the schema directly:

```
CREATE TABLE Enrolment (
    enrolment_id INT PRIMARY KEY,
    student_id   INT REFERENCES Student(student_id),
    course_id    INT NOT NULL REFERENCES Course(course_id),
    enrol_date   DATE
);
```

No `NOT NULL` on `student_id`. No `ON DELETE CASCADE`. When a student withdraws, **847 enrolment records survive as orphans** — "ghost enrolments" that inflate class sizes and corrupt reporting.

The business rule (mandatory participation, cascade delete) was never captured in a design artefact before committing to tables.

The Sharp Question

Neither failure is a bug. The code ran. The database accepted the data.

	ShopEasy	UniTrack
Symptom	Ambiguous cardinality — two code paths, no way to tell which was intended	Ghost enrolments inflate class sizes
Root cause	FK exists but no NOT NULL, no UNIQUE — cardinality undocumented	Nullable `student_id`, no ON DELETE CASCADE
Gap	Requirements → DDL with nothing in between	Same — no intermediate artefact

> Therefore, the designer must **capture business rules in a readable artefact before committing to tables** — not discover ambiguities in production.

That artefact is the **Entity-Relationship Diagram (ERD)**.

Five Requirements for a Design Tool

Each requirement maps to a failure we just saw:

1. **Capture cardinality explicitly** — ShopEasy's "one or many payments?" was unanswerable. *Test: every relationship has a (min, max) pair on each side.*

2. **Make translation deterministic** — ShopEasy's constraints were guessed. *Test: given ERD, two developers produce the same schema.*

3. **Support round-trip verification** — UniTrack's cascade rule was missing. *Test: walk every (min, max) → confirm matching constraint.*

4. **Handle structural patterns** — UniTrack's junction table was incomplete. *Test: M:N, weak entity, self-reference all representable.*

5. **Be cheap to change** — both failures happened in production, not design. *Test: changes to ERD precede changes to schema.*

Two Notations, One Discipline

	Chen notation	Crow's foot notation
Role	Thinking tool — design and review	Deliverable — implementation blueprint
Shapes	Rectangles, diamonds, ovals, (min, max) pairs	Table boxes, FK lines, cardinality symbols
Audience	Business stakeholders + designers	Developers + database tools
Tools	Whiteboard, ERDPlus (Chen view)	dbdiagram.io, MySQL Workbench, ERDPlus (relational view)

Throughout this topic: **Chen** for design, **crow's foot** for schema.

> **Predict:** Is an entity the same thing as a table?

Entity ≠ Table

An **entity** is a conceptual construct — a thing in the business domain you choose to track.

A **table** is its physical implementation in a relational schema.

An entity *becomes* a table when you translate the ERD to a schema (Step 6). The distinction matters because you design at the entity level — "what business thing are we modelling?" — not "what columns do we need?"

The table follows mechanically from the entity definition plus the derivation rules.

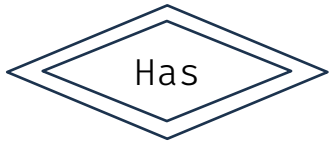
The Seven Chen Shapes

Every Chen ERD is built from seven shapes — nothing else:

Strong Entity

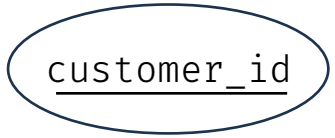
Weak Entity – Double Border

Regular Relationship

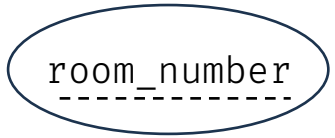


Identifying Relationship

Regular Attribute



Key Attribute – Underline

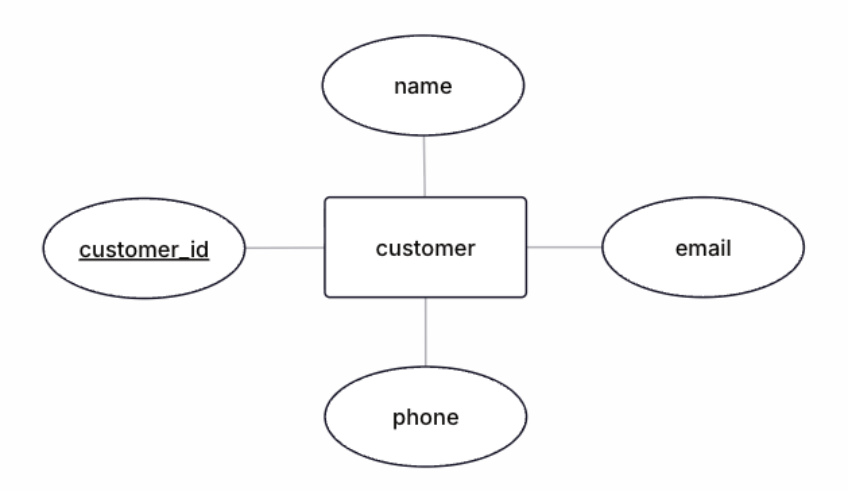


Partial Key – Dashed Line

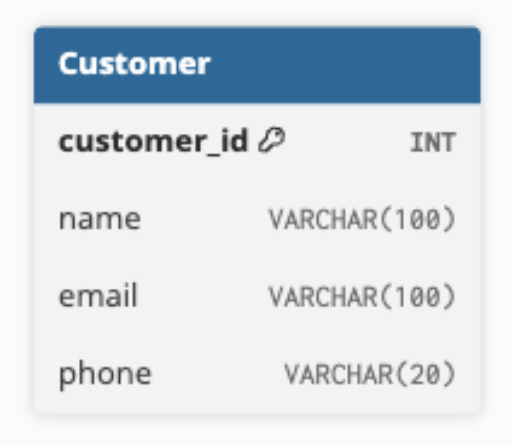
Key attribute (solid underline) uniquely identifies a strong entity. **Partial key** (dashed underline) is unique only within one owner instance.

Entities and Attributes — The Balloon Diagram

In Chen notation, each attribute is an oval connected to its entity rectangle:



One entity, one table. Every oval becomes a column. The underlined key becomes the PK



Relationships and (min, max) Cardinality

A **relationship** is an association between two entities — drawn as a diamond in Chen notation.

Each side carries a **(min, max)** pair:

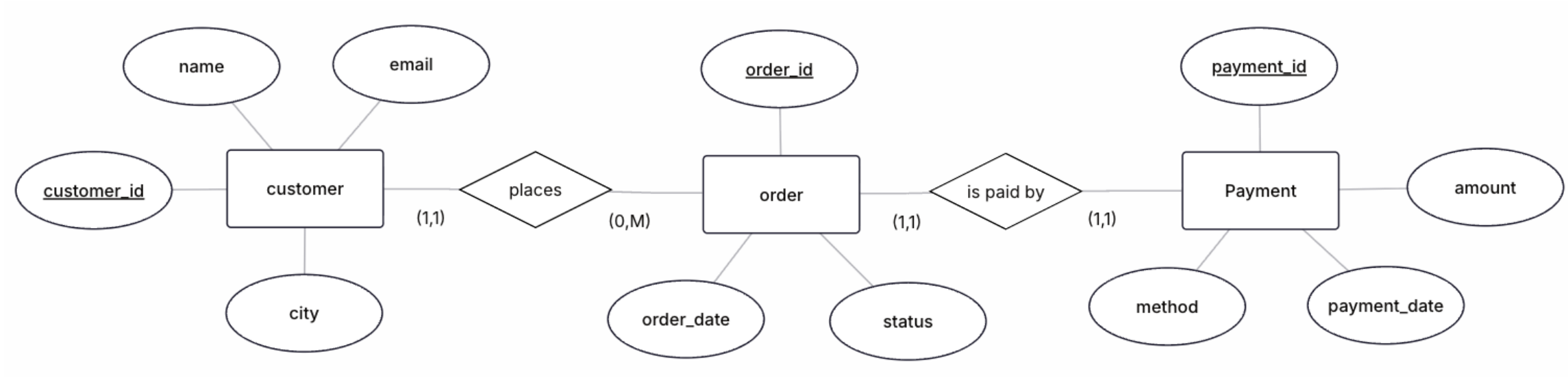
- **min** ∈ {0, 1} — participation: 0 = optional, 1 = mandatory (total)
- **max** ∈ {1, N} — cardinality: 1 = at most one, N = unbounded

(min, max)	Meaning	Plain English
(1, 1)	Exactly one, mandatory	Every `order` is placed by exactly one `customer`
(0, 1)	At most one, optional	An `order` may have at most one `shipment` record
(1, N)	One or more, mandatory	Every `order` includes at least one `product`
(0, N)	Zero or more, optional	A `customer` may place zero or more orders

Reading (min, max) — Read the Contract

To read the (min, max) near entity B: "**For one instance of A, what are the minimum and maximum number of B instances?**"

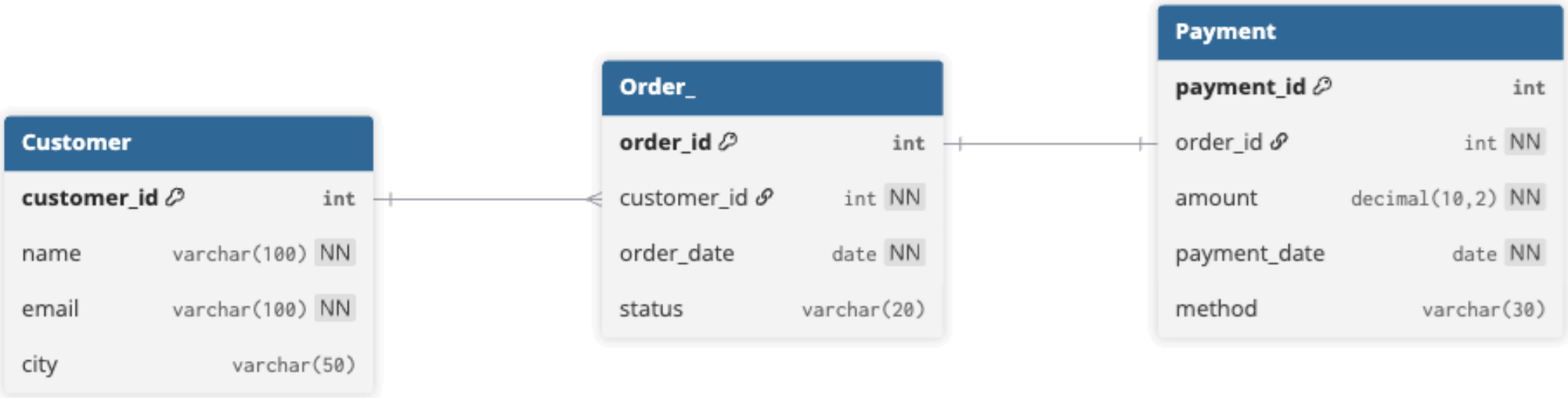
Stand at one entity, read the label near the other:



- Stand at `customer`, read near `order`: **a customer places 0 to N orders**
- Stand at `order`, read near `customer`: **each order is placed by exactly 1 customer**

The Same Fragment in Crow's Foot

The same business rules, now as a relational schema diagram:



Crow's foot symbols: fork (<) = many, bar (|) = one, circle (○) = optional, double bar (||) = mandatory.

The Cardinality-to-Constraint Mapping

This is the **single most important reference** in this topic — the bridge between the ERD and the schema:

Entity's (min, max)	Crow's foot	NOT NULL?	UNIQUE on FK?	Pattern
(1, 1)	‖—┤ mandatory, one	Yes	Yes	Exactly one; mandatory
(0, 1)	○—┤ optional, one	No	Yes	At most one; optional
(1, N)	‖—< mandatory, many	Yes	No	One or more; mandatory
(0, N)	○—< optional, many	No	No	Zero or more; optional
M:N	>—< many-to-many	Both FKs NOT NULL	Composite PK	Junction table

The **min** drives NOT NULL. The **max** drives UNIQUE or FK placement.

> **Predict:** ShopEasy's ERD says `payment` (1,1) — is paid by → (1,1) `order`. What constraints should `order_id` in `payments` have?

ShopEasy's Missing Constraints — Revealed

NOT NULL (min = 1, mandatory) **and** UNIQUE (max = 1, at most one payment per order).

That's a 1:1 relationship. The mapping table produces it mechanically.

ShopEasy's original schema had the FK but lacked NOT NULL and UNIQUE — the cardinality was undocumented. The mapping table would have made the required constraints explicit in five minutes at a whiteboard.

This is the core promise: the ERD commits to the business rule. The mapping translates it to constraints. The schema enforces them. **No judgement calls. No gaps.**

Three Structural Patterns — Overview

Beyond basic 1:N, three patterns arise regularly:

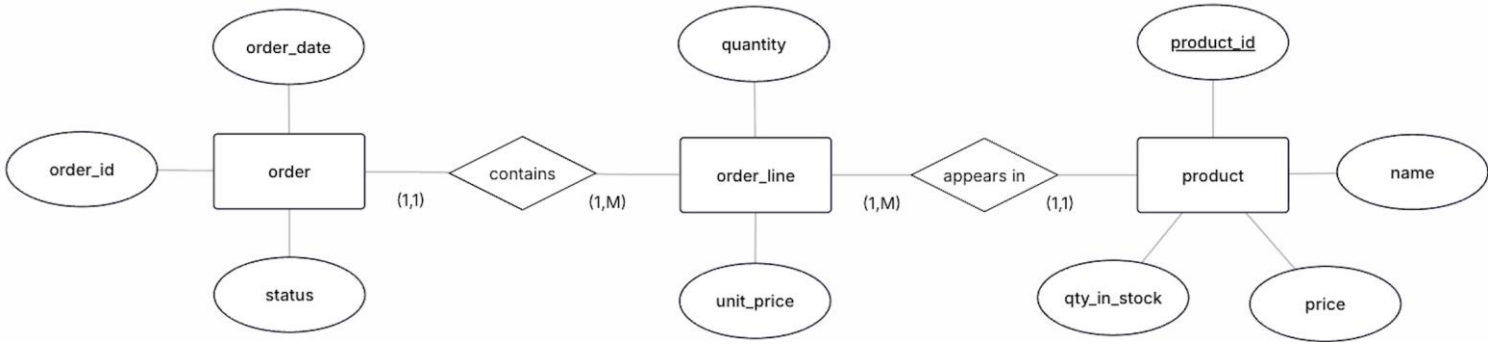
Pattern	When it arises	What's distinctive
Associative entity	M:N relationship carries its own attributes	Junction table with composite PK
Weak entity	Entity's identity depends on its owner	Composite PK = owner FK + partial key
Self-reference	Entity related to itself	FK references own table

Each follows the same four derivation rules — different (min, max) inputs produce different structural consequences.

Associative Entity — `order` includes `product`

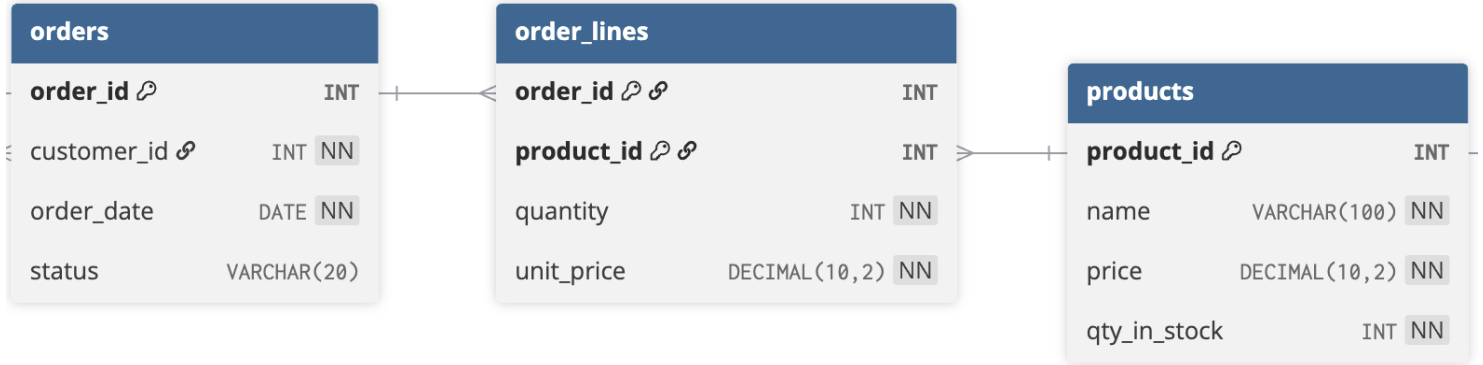
The relationship between `order` and `product` is M:N — and it carries `quantity` and `unit_price`. Those belong to the *pairing*, not to either entity alone.

Equivalence rule: an M:N relationship that carries its own attributes is decomposed into two 1:M relationships via an **associative entity** — a new entity (rectangle) that replaces the original relationship diamond.



The M:N diamond is gone. `order_line` is now a regular entity connected by two 1:M relationships — same seven shapes, no special notation. It becomes the junction table:

Composite PK `(order_id, product_id)` prevents duplicates.



> **Predict:** What happens if a customer adds 3 units of the same product, then later adds 2 more? Can the composite PK handle this?

Associative Entity — No Free Lunch

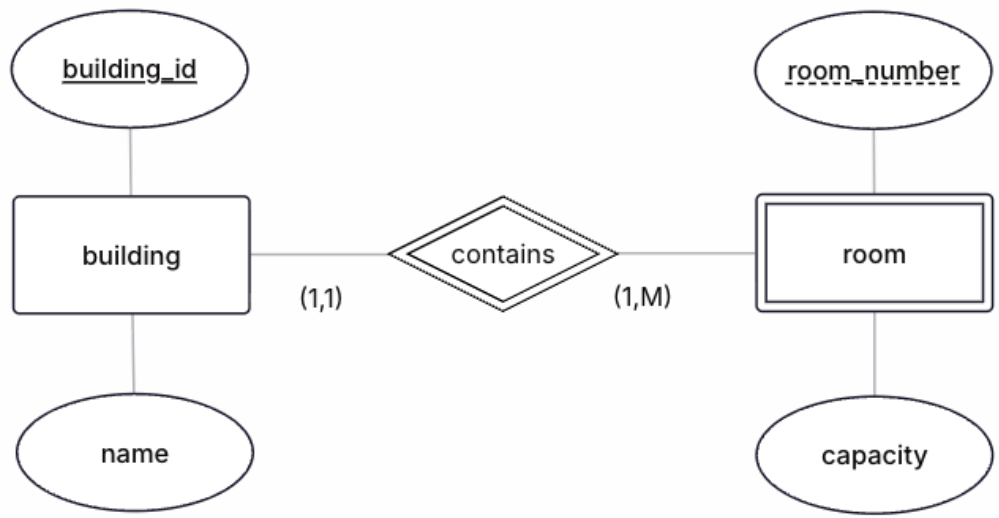
No — the composite PK enforces uniqueness on the pair. A second row with the same `(order_id, product_id)` violates the PK constraint.

The application must either **update the existing row's quantity** (3 → 5) or the composite key must include a differentiating attribute (e.g., a line number).

The ERD forces you to confront this decision **before writing code**. No free lunch — the composite PK gives you uniqueness, but it constrains how duplicates are handled.

Weak Entity — `room` Within `building`

A **weak entity** can't be uniquely identified by its own attributes. Room 101 in Building A ≠ Room 101 in Building B.



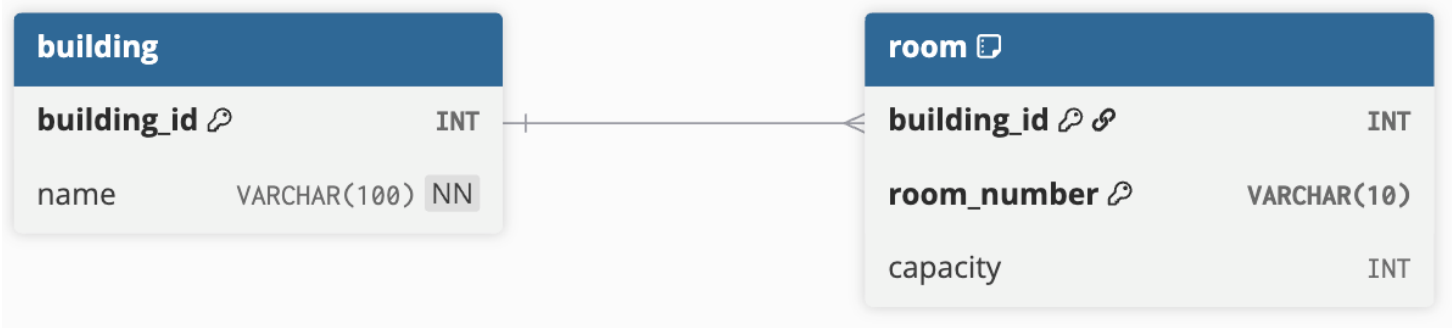
Three visual cues: **double rectangle** (weak entity), **double diamond** (identifying relationship), **dashed underline** (partial key).

> **Predict:** What is `room`'s primary key? What happens when a `building` is deleted?

Weak Entity — Schema Consequences

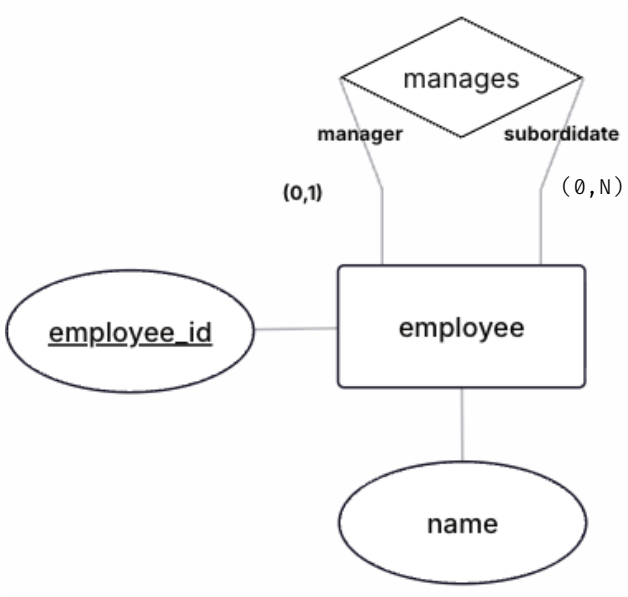
Two consequences in the schema:

1. **Composite PK** — `(building_id, room_number)`: the owner's FK plus the partial key
2. **CASCADE delete** — delete the building, delete all its rooms



Self-Reference — `employee` Hierarchy

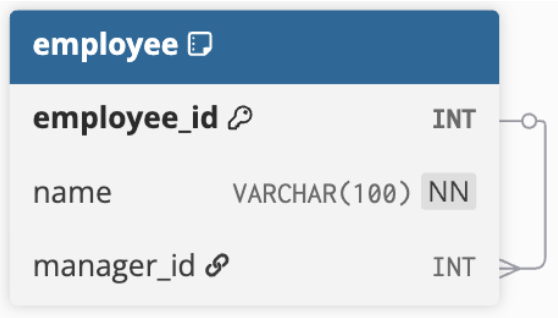
An entity related to itself. The classic: an `employee` may be managed by another `employee`.



Role labels are essential — without them, you can't tell which side is which.

> **Predict:** The CEO has no manager. Some managers manage many subordinates. Is `manager_id` nullable or NOT NULL? Is it UNIQUE?

Self-Reference — Schema Consequences



Nullable (min = 0 — CEO has no manager). **Not UNIQUE** (max = N — one manager, many subordinates).

The FK loops back to the same table. The derivation rules apply identically — the self-reference changes nothing about how (min, max) translates to constraints.

Four Patterns — Which Rules Fire?

Pattern	Example	Rules that fire	Structural result
1:N (already seen)	`customer` → `order`	Rule 1 (FK on N-side), Rule 2 (NOT NULL)	FK column on child table
M:N (associative)	`order` ↔ `product`	Rule 4 (junction table), Rule 2 (both NOT NULL)	Junction table with composite PK + attributes
Weak entity	`building` → `room`	Rule 4 (composite PK), Rule 2, CASCADE	PK = owner FK + partial key
Self-reference	`employee` → `employee`	Rule 1 (FK on N-side), Rule 2 (nullable)	FK references own table

Same four rules. Different (min, max) inputs produce different structural consequences. The rules don't change — the ERD tells them what to produce.

Why Two Levels?

Redrawing a cardinality on an ERD takes **seconds**. Altering a production schema with 1.4 million rows takes a **migration, downtime, and risk**.

The ERD is where you make mistakes cheaply — *before* committing to tables.

But two levels create a risk: **drift**. The developer adds a FK the ERD doesn't show. The ERD says (1,1) but the schema allows NULL.

Drift is how business rules get lost — the same failure pattern from ShopEasy.

The antidote: **round-trip verification** (Step 7 of the procedure).

The Four Derivation Rules

These four rules make the entire ERD-to-schema translation **deterministic**:

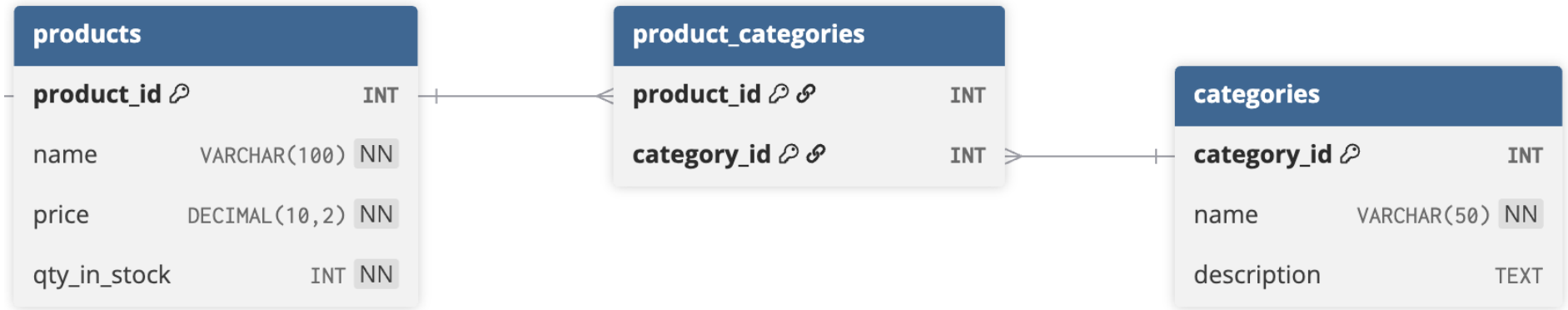
Rule	Condition	Action
Rule 1 — FK placement	1:N relationship	FK on the N-side table
	M:N relationship	Junction table holds both FKs
	1:1 relationship	FK on the mandatory side
Rule 2 — NOT NULL	min = 1 (mandatory)	FK column is NOT NULL
	min = 0 (optional)	FK column is nullable
Rule 3 — UNIQUE on FK	Parent's max toward FK-bearer = 1	UNIQUE on FK (enforces 1:1)
	Parent's max = N	No UNIQUE
Rule 4 — Composite PK	M:N junction table	PK = both FKs
	Weak entity	PK = owner FK + partial key

Given any (min, max) pair, the derivation is mechanical.

> **Predict:** `product` (0,N) ← categorised as → (0,N) `category` — M:N. What does the schema look like?

M:N Derivation — Revealed

Rule 4 fires: junction table `product_categories` with composite PK `(product_id, category_id)`.



Both FKs are NOT NULL (they form the PK). No UNIQUE on either — many-to-many means multiple rows per parent.

Scope Note — What We Defer to Topic 5

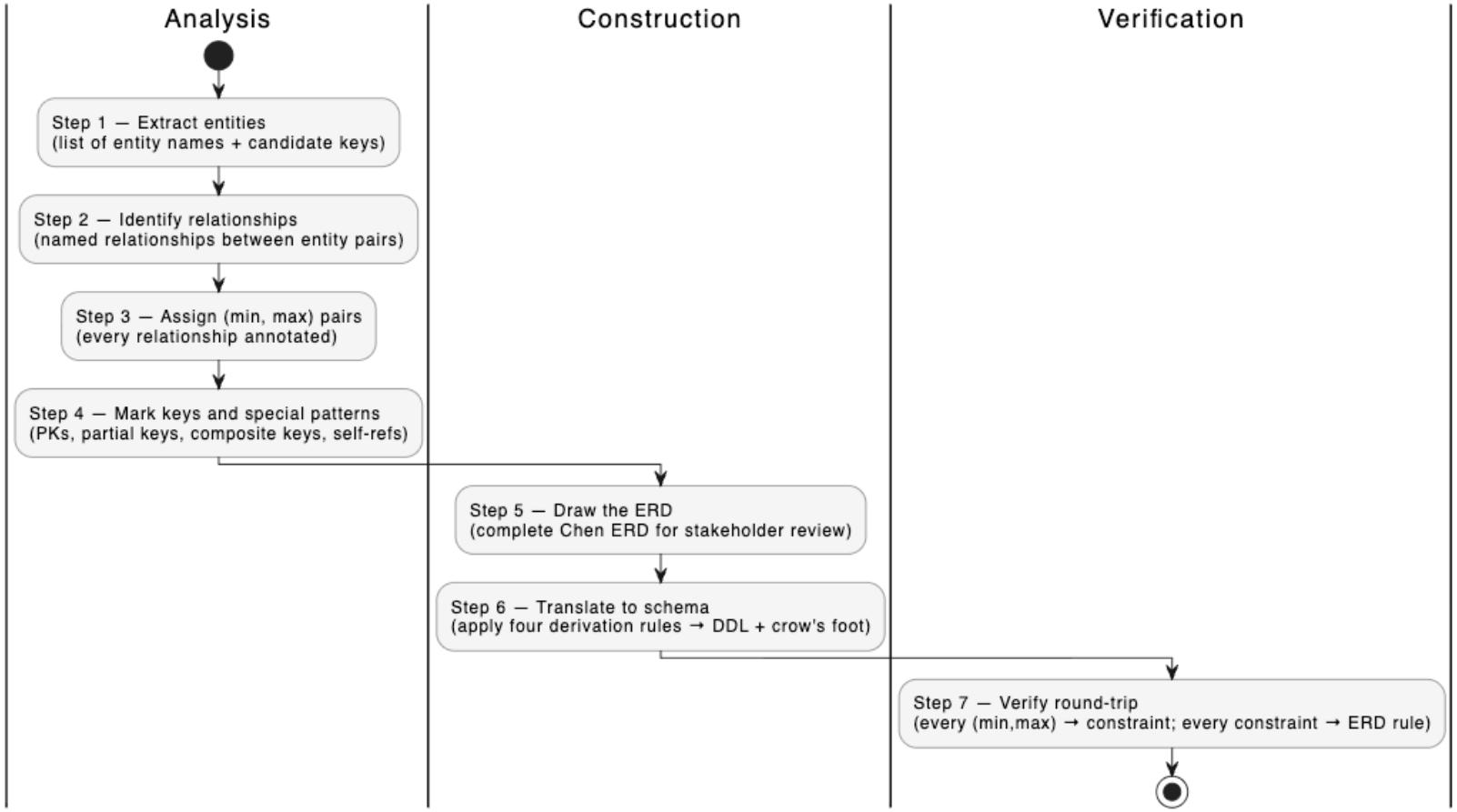
The four derivation rules produce a **structurally correct schema**: tables, PKs, FKs, NOT NULL, UNIQUE.

A production-ready schema also needs:

- CHECK constraints for domain rules (`quantity > 0`)
- CASCADE behaviours (RESTRICT vs CASCADE vs SET NULL)
- Indexes for performance
- Views for access control
- Transactions for atomicity

On Topic 1's enforcement spectrum, the ERD's (min, max) pairs produce the "**engine rejects**" constraints. **Topic 5** completes the picture.

The Seven-Step Procedure



Steps 1–4: analysis. Steps 5–6: construction. **Step 7: the acceptance test ShopEasy never had.**

Worked Example — ShopEasy Domain Description

ShopEasy manages customers, orders, products, payments, shipments, reviews, suppliers, and categories.

- One customer places zero or more orders
- Each order contains one or more products (with quantity and unit price)
- Each order is paid by exactly one payment (no instalments)
- Each order may optionally be shipped
- Customers write reviews for products
- Products are supplied by one or more suppliers
- Each product belongs to one or more categories

We'll apply all seven steps to this domain.

Steps 1–2: Entities and Relationships

Eight entities: `customer`, `order`, `product`, `payment`, `shipment`, `review`, `supplier`, `category`

Eight relationships:

Relationship	Entities	Verb phrase
places	`customer` → `order`	`customer` places `order`
includes	`order` ↔ `product`	`order` includes `product` (with qty, price)
is paid by	`order` → `payment`	`order` is paid by `payment`
is shipped by	`order` → `shipment`	`order` is shipped by `shipment`
writes	`customer` → `review`	`customer` writes `review`
reviews	`review` → `product`	`review` reviews `product`
supplied by	`supplier` ↔ `product`	`product` supplied by `supplier`
categorised as	`product` ↔ `category`	`product` categorised as `category`

Predict: The business rule says "no instalments — each order is paid by exactly one payment." What should the (min, max) pairs be for `is paid by`?

Step 3: Assign (min, max) Pairs

Relationship	Side A	(min, max)	Side B	(min, max)	Type
places	`customer`	(0, N)	`order`	(1, 1)	1:N
includes	`order`	(1, N)	`product`	(0, N)	M:N
is paid by	`order`	(1, 1)	`payment`	(1, 1)	1:1
is shipped by	`order`	(0, 1)	`shipment`	(1, 1)	1:1
writes	`customer`	(0, N)	`review`	(1, 1)	1:N
reviews	`product`	(0, N)	`review`	(1, 1)	1:N
supplied by	`supplier`	(0, N)	`product`	(1, N)	M:N
categorised as	`category`	(0, N)	`product`	(1, N)	M:N

Notice: is paid by is (1,1)↔(1,1) — the "no instalments" rule. is shipped by is (0,1)↔(1,1) — shipment is conditional. **One digit different in (min, max) → different schema constraints.**

Step 4: Mark Keys and Special Patterns

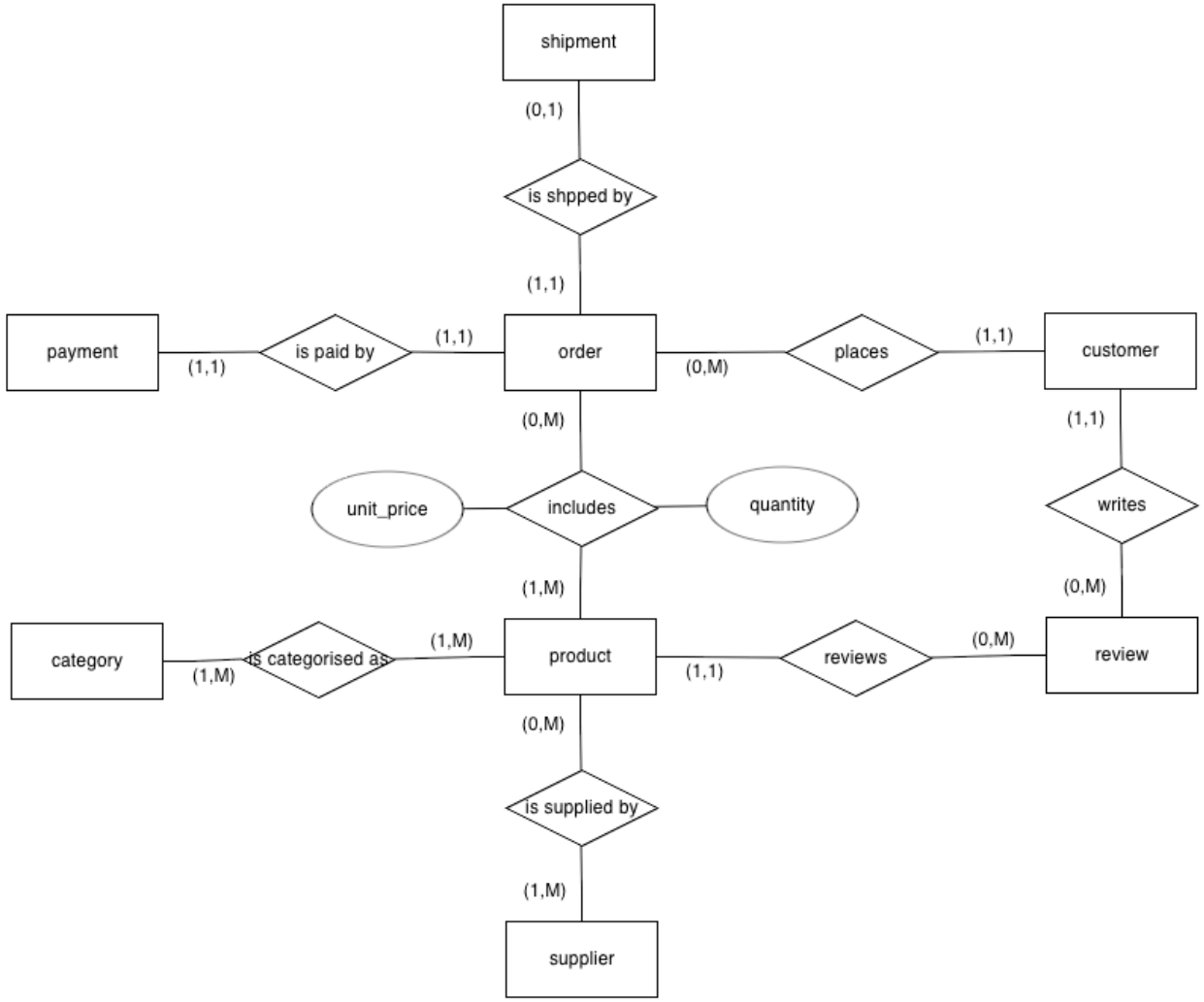
One structural pattern identified:

- **Associative entity:** `order_line` — the includes relationship carries `quantity` and `unit_price`. Composite key: `(order_id, product_id)`.

No weak entities in this domain — all entities have independent keys. No self-references — the employee hierarchy appears in the lab.

Step 4 is where you decide: does any relationship need a junction table, a composite PK, or a self-referencing FK? Here, only includes qualifies.

Step 5: The Complete ERD



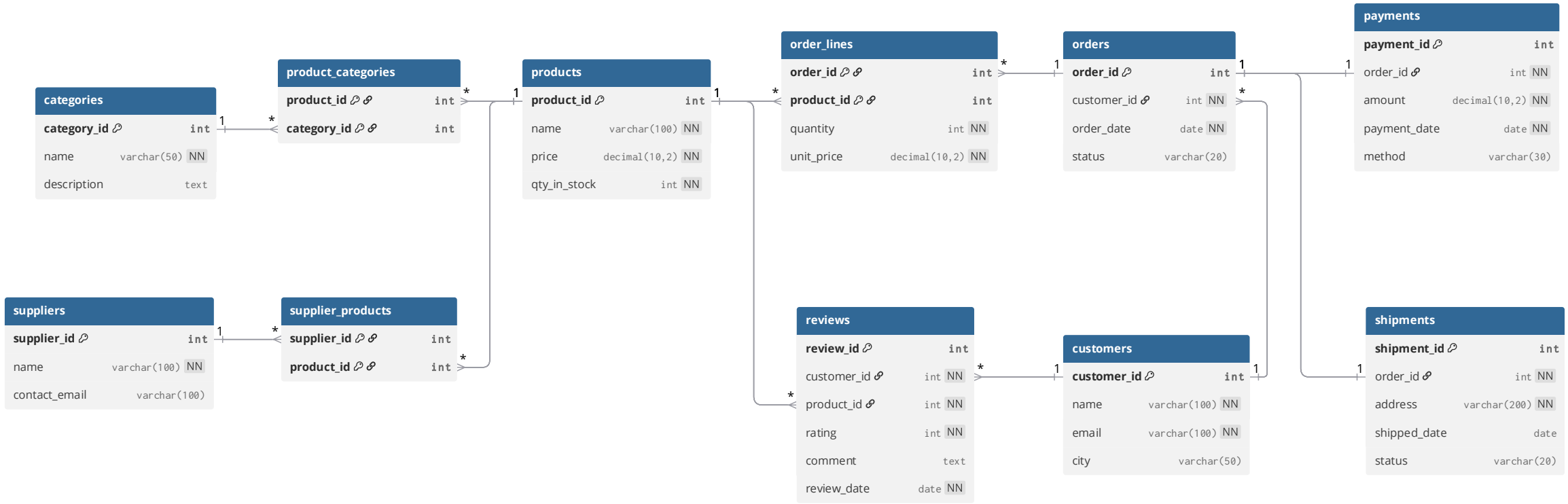
Every constraint in the schema traces back to a (min, max) pair in this diagram.

Step 6: Derivation Trace

Relationship	Type	FK placement	NOT NULL	UNIQUE	Composite PK
places	1:N	`customer_id` on `orders`	Yes (min=1)	No (max=N)	—
includes	M:N	Junction: `order_lines`	Both NOT NULL	No	PK(order_id, product_id)
is paid by	1:1	`order_id` on `payments`	Yes (min=1)	Yes (max=1)	—
is shipped by	1:1	`order_id` on `shipments`	Yes (min=1)	Yes (max=1)	—
writes	1:N	`customer_id` on `reviews`	Yes (min=1)	No (max=N)	—
reviews	1:N	`product_id` on `reviews`	Yes (min=1)	No (max=N)	—
supplied by	M:N	Junction: `supplier_products`	Both NOT NULL	No	PK(supplier_id, product_id)
categorised as	M:N	Junction: `product_categories`	Both NOT NULL	No	PK(product_id, category_id)

Same four rules, applied eight times. The derivation is mechanical.

Step 6: The Complete Schema



Step 7: Round-Trip Verification

ERD rule	Expected constraint	Present?
`order` → `customer`: (1,1)	`customer_id` NOT NULL, no UNIQUE	Yes
`customer` → `order`: (0,N)	No constraint (zero or more)	Yes
`order` ↔ `product`: M:N	Junction table with composite PK	Yes (`order_lines`)
`order` → `payment`: (1,1)	`order_id` NOT NULL UNIQUE	Yes
`order` → `shipment`: (0,1)	`order_id` NOT NULL UNIQUE	Yes
`review` → `customer`: (1,1)	`customer_id` NOT NULL	Yes
`review` → `product`: (1,1)	`product_id` NOT NULL	Yes
`product` ↔ `supplier`: M:N	Junction with composite PK	Yes (`supplier_products`)
`product` ↔ `category`: M:N	Junction with composite PK	Yes (`product_categories`)

All nine relationships verified. No gaps. **This five-minute check is what would have caught both ShopEasy's ambiguity and UniTrack's missing cascade.**

> **Predict:** `shipment` is optional — `order` (0,1) is shipped by (1,1) `shipment`. But `order_id` in `shipments` is NOT NULL. Where does the optionality live?

Optionality Lives in Row Absence

```
orders                shipments
order_id  status      order_id  address
────────  ──────      ────────  ───────────
1001      paid  ←── 1001      12 High St
1002      paid  ←── 1002      8 Park Ave
1003      paid      (no row) ← not shipped
```

Order 1003 has no shipment row — and that absence **is** the optionality. The NOT NULL on `order_id` means: *if a shipment record exists, it must reference an order*. It does not force every order to have a shipment record.

The UNIQUE constraint ensures at most one shipment per order. **Absence of a row** is the encoding of "not shipped."

A common mistake: assuming min = 0 always means a nullable FK. In a 1:1 where the FK is on its own table, optionality is encoded by **presence or absence of the entire row**.

Check 1 — Doctor-Patient (Forward Translation)

A hospital ERD: Doctor (0, N) ← treats → (1, N) Patient. Each patient is treated by at least one doctor; a doctor may treat zero or more patients.

Predict: What schema structure results?

M:N → junction table `DoctorPatient`. Rule 4: composite PK `(doctor_id, patient_id)`. Both FKs NOT NULL. No UNIQUE on either individual FK — that would collapse M:N back to 1:1. A patient who sees three doctors produces three rows, each with a different `doctor_id`.

Check 2 — Cardinality Change (What Breaks)

Same hospital. New policy: each patient must have exactly **one primary doctor**. The relationship changes from M:N to 1:N: Doctor (0, N) ← has primary → (1, 1) Patient.

Predict: How does the schema change?

The junction table **disappears**. A `primary_doctor_id FK NOT NULL` column appears on Patient (Rule 1: N-side). No UNIQUE (max = N from Doctor side). A table was deleted, a column was added, constraints changed — from **one cardinality change**. That's why you commit to cardinalities in the ERD first.

Check 3 — Drift Detection (Library)

A Library ERD: Member (0, N) ← borrows → (0, 5) Book. The schema:

```
CREATE TABLE Loan (
    member_id INT NOT NULL REFERENCES Member(member_id),
    book_id   INT NOT NULL REFERENCES Book(book_id),
    PRIMARY KEY (member_id, book_id)
);
```

A developer notices: nothing limits a member to 5 concurrent loans. Is this a bug?

> This is **drift** — the ERD says max = 5, but FK/PK/UNIQUE constraints can't express max = 5. That needs a CHECK constraint or trigger (Topic 5). The round-trip check surfaces the gap.

Checks 1–3: The Pattern So Far

Check	What it tests	Lesson
Doctor-Patient	Forward M:N translation	Same four rules, mechanical derivation
Cardinality change	One digit changes → schema restructures	Commit to cardinalities in the ERD first
Library drift	ERD vs schema disagreement	Round-trip verification catches gaps

Three checks — forward translation, structural impact, and drift detection. The remaining checks test **harder cases**: weak entities, missing ERDs, and cross-topic reasoning.

Check 4 — Weak Entity (Parking Garage)

Garages contain Levels; Levels contain Spaces. Both ParkingLevel and ParkingSpot are weak entities.

Predict: What are the primary keys for ParkingLevel and ParkingSpot?

ParkingLevel: PK `(garage_id, level_number)`. ParkingSpot depends on ParkingLevel, which depends on ParkingGarage — the identity chain cascades. ParkingSpot: PK `(garage_id, level_number, spot_number)`. **Three columns** — the full key of the deepest weak entity includes all ancestor keys. ON DELETE CASCADE at both levels.

Check 5 — Missing ERD (Task Management)

A startup writes this schema directly — no ERD:

```
CREATE TABLE Task (
    task_id     INT PRIMARY KEY,
    assignee_id INT REFERENCES User(user_id),
    project_id  INT REFERENCES Project(project_id)
);
```

Three questions arise: (a) Can a task have no assignee? (b) Can a task belong to multiple projects? (c) Can one user be assigned to the same task twice?

> (a) `assignee_id` has no NOT NULL — so yes. But was this intentional or an oversight? (b) `project_id` is a single FK — one task, one project. If the business wants multi-project tasks, the schema is wrong (needs a junction table). (c) One `assignee_id` per task, but what about *multiple* assignees? Without an ERD, the schema can't distinguish "we decided" from "we forgot."

Check 6 — Cross-Topic (Shape Alignment)

Checks 1–5 assumed the relational model was the right choice. This check asks a different question: **what if it isn't?**

A mobile app loads a product with all its reviews in one API call. The delivery shape is nested JSON:

```
{ "product": "...", "reviews": [{"rating": 5, "comment": "..."}, ...] }
```

The ERD produces flat, normalised tables: `products` and `reviews` linked by a FK. To deliver that nested JSON, the application must JOIN the tables, then nest the results in code. This is the **same shape mismatch** from Topic 1's RideLink failure.

> The ERD-to-schema pipeline produces a correct **relational** schema. Whether relational is the right **model** for the workload is a separate question — one Topic 1's framework answers. If aggregate reads dominate, the document model may fit better. If ACID is also required, the signals conflict — Topic 11 territory.

The ERD is the relational instantiation, but the principle — **capture the rules before you build** — is universal, whatever the model.

Checks 4–6: Harder Cases

Check	What it tests	Lesson
Parking garage	Cascading weak entity identity	Full key = all ancestor keys
Task management	What happens without an ERD	Schema becomes ambiguous record of intent
Shape alignment	Cross-topic: ERD ≠ model selection	ERD shapes relational storage; Topic 1 shapes model choice

Six checks — from forward translation to cross-topic reasoning. The common thread: the ERD-to-schema pipeline catches structural errors mechanically, but the designer still decides **which model**, **which constraints**, and **which trade-offs** to encode. The pipeline removes guesswork; it doesn't remove judgement.

Callback: The Sharp Question Answered

Earlier we asked: **if you can't get from requirements to DDL without dropping constraints, what artefact should sit between them?**

We can answer it fully now.

ShopEasy's missing NOT NULL and UNIQUE → the cardinality-to-constraint mapping catches those mechanically. If the ERD says (1,1) ↔ (1,1), the mapping produces NOT NULL + UNIQUE. The FK existed, but without UNIQUE the cardinality was undocumented — was it 1:1 or 1:N? The mapping table removes the ambiguity.

UniTrack's ghost enrolments → Step 7 round-trip verification catches it. ERD says min = 1 (mandatory). Schema has nullable FK. They disagree. Fix it. The structural translation doesn't produce ON DELETE CASCADE — that’s Topic 5's enforcement layer — but round-trip verification makes the gap visible so it can be addressed before deployment.

Both failures are now diagnosable using tools from this topic. The ERD is the missing artefact. The seven-step procedure is the process. Round-trip verification is the safety net.

Course Map

```
Topic 1 (vocabulary, decision framework, four models + validation surveyed)
    │
    ├── Topics 2–5: Relational deep-dive
    │   ├── ▸ Topic 2: ERD — capturing business rules before committing to tables
    │   ├── Topic 3: Normalisation — eliminating redundancy; BCNF vs star schema
    │   ├── Topic 4: SQL — querying with JOINs, aggregation, CTEs
    │   └── Topic 5: Enforcement — constraints, transactions, indexes, access control
    │
    ├── Topics 6–10: Alternative models + cross-cutting discipline
    │   ├── Topic 6: Document (MongoDB) — revisits RideLink profiles
    │   ├── Topic 7: Schema validation — XSD and JSON Schema at trust boundaries
    │   ├── Topic 8: RDF — knowledge graphs; global identifiers; open-world assumption
    │   ├── Topic 9: SPARQL — querying knowledge graphs (builds on Topic 8)
    │   └── Topic 10: Property graph (Neo4j/Cypher) — bounded traversal
    │
    └── Topic 11: Capstone — multi-model design (all four models + validation, one system)
```

The schema produced in Topic 2 is the input for Topics 3–5.

Concepts That Recur

Six ideas planted here — each appears briefly now and grows into a full mechanism later:

Concept	Introduced here as	Fully developed in
(min, max) cardinality	ERD cardinality pairs	Topic 7 (XSD `minOccurs`/`maxOccurs`)
Crow's foot notation	Relational schema diagrams	Lab (dbdiagram.io); industry tools
Associative entity	Junction table for M:N with attributes	Topic 6 (nested array), Topic 8 (RDF reification), Topic 10 (edge properties)
Round-trip verification	ERD ↔ schema constraint mapping	Topic 7 (XSD ↔ instance validation)
Read the contract	Reading (min, max) pairs	Topic 7 (element cards), Topic 8 (property cardinalities)
Cascade delete	ON DELETE CASCADE on weak/dependent entities	Topic 5 (FK cascade behaviours: RESTRICT, CASCADE, SET NULL)

The (min, max) reading skill transfers directly: `minOccurs`/`maxOccurs` in XSD, property cardinalities in RDF. Learn it once — it pays off three times.

Next: Topic 3 — Normalisation

The schema produced here is **structurally correct** — every business rule from the ERD is enforced.

But is it free of **redundancy**? What if the same fact is stored in two places — and an update changes one but not the other?

Topic 3 introduces **functional dependencies** and **normal forms** (BCNF) as diagnostic tools for detecting and eliminating redundancy in the schema we just built.

Lab Preview

Two-part lab — ERD construction + schema translation.

- **Part 1 — ERD construction (in-class, ERDPlus):** Build a Chen ERD from a domain description using the seven-step procedure. Incrementally add entities, relationships, and (min, max) pairs.
- **Part 2 — Schema translation (dbdiagram.io → MySQL):** Translate the ERD to a relational schema. Export DDL. Verify round-trip: walk every (min, max) pair and confirm the matching constraint.

Lab file: `02-lab-erd-to-schema.md`